

LECTURE 1

Introduction to Databases and Information Systems

CSC270 · Database Systems

COMSATS University Islamabad, Lahore Campus

In This Lecture

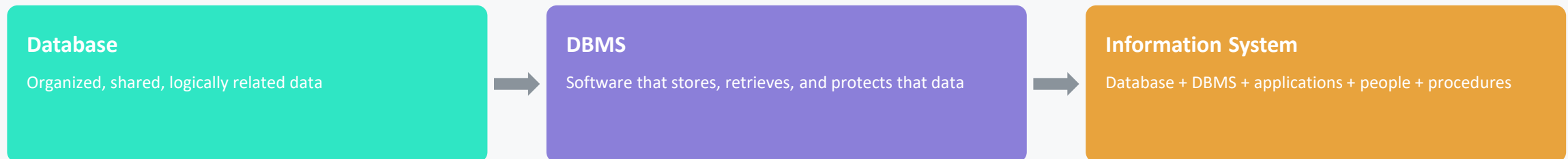
- What a database is, and how it differs from an information system built around one
- Common categories of database applications you already use
- How data management evolved from manual filing to file-based systems to databases
- File-based systems: how they worked, and the problems that forced a rethink
- The database approach: what changes, concretely, when data is centralized
- The advantages a DBMS gives you — and the real costs it imposes in return

Databases and Information Systems

A **database** is a shared, organized collection of logically related data, designed to meet an organization's information needs.

- Shared — many users and programs draw on the same data, not private copies
- Organized — structured so it can be found, related, and reasoned about
- Logically related — records about the same real-world entity are known to be connected

From data to an information system



Database Applications

Domain	What's stored	Example
Banking	Accounts, balances, transactions	Approving a debit card transaction
Airlines	Reservations, schedules, seat maps	Confirming a seat on flight PK-301
University records	Students, courses, enrollments, grades	Producing a graduate's transcript
Telecom	Call records, subscriber accounts	Generating a monthly phone bill
Sales / e-commerce	Products, orders, customers, inventory	"Only 3 left in stock"
Human resources	Employees, salaries, positions	Running monthly payroll

A quick test: is this really a database problem? If the data must be shared by more than one person or program, stay accurate under concurrent access, and survive a crash without loss — it belongs in a database, not a private file.

Evolution of Data Management

Data management evolved in stages, each responding to the weaknesses of the one before it.

Manual / paper filing

Ledgers, index cards, filing cabinets — slow, error-prone, one copy



File-based systems

Each application owns its own files; no coordination between them



Database approach (DBMS)

One managed, shared collection of data serving every application

File-Based Systems: A University Before Databases

Suppose COMSATS ran three separate offices, each with its own file-based system, all needing student data:

Registrar's file

RegNo, Name, CNIC, Address, Program

Library's file

RegNo, Name, Address, BooksIssued

Finance office's file

RegNo, Name, Address, FeeBalance

Name and Address are stored three separate times — once per office, each written by its own program. That single decision is the root of almost every problem file-based systems became known for.

- Data redundancy — the same facts duplicated across every file that needs them
- Data inconsistency — independent copies drift apart with no way to tell which is current
- Poor data sharing — files were never designed to talk to each other
- Program-data dependence — physical file layout is hard-coded into each program

File-Based Systems: Why It Broke Down

- Limited sharing, unanticipated queries — “students who owe a library fine AND a tuition balance” spans two incompatible files; answering it means writing a brand-new program
- Integrity and security are hard to enforce consistently — each program enforces its own rules (or doesn't); there's no single place to say “a fee balance can never be negative”

Inconsistency is the most dangerous failure mode — not the most obvious one

Wasted storage from redundancy is a cost you can measure and budget for. Inconsistency is worse precisely because it's invisible until it causes damage — a graduating student blocked from receiving their transcript because the finance office's stale copy of their fee balance still shows an unpaid amount that was actually settled weeks earlier, according to the registrar's copy.

The Database Approach

Instead of every application owning a private copy of the data it needs, **store the data once, centrally, and let every application go through a DBMS to reach it.**

File-based (before)

Registrar, Library, and Finance each keep their own Name/Address copy — three files, three truths

Database approach (after)

One Student table holds Name/Address once; every office reads the same row through the DBMS

Worked example: a single Student relation

RegNo	Name	Address	Program
FA21-BCS-001	Ali Raza	Model Town, Lahore	BS Computer Science

The library's and finance's systems no longer keep their own Name/Address copies — they store only BooksIssued / FeeBalance and reference the same shared row through the DBMS.

Advantages of the Database Approach (1/2)

- **Control of data redundancy** — store each fact once wherever possible
- **Data consistency** — with redundancy controlled, copies can't disagree
- **More information from the same data** — combining files answers questions no single file could
- **Sharing of data** — existing data can be reused by new applications
- **Improved data integrity** — validation rules are defined once and enforced everywhere
- **Improved security** — access restricted per-user, down to individual columns

Advantages of the Database Approach (2/2)

- **Enforcement of standards** — date and ID formats enforced centrally, not per-programmer
- **Economy of scale** — one managed system costs less overall than many small ones
- **Balance of conflicting requirements** — a DBA arranges data for the organization as a whole
- **Improved accessibility and responsiveness** — query languages answer unanticipated questions directly
- **Increased productivity** — the DBMS provides storage, retrieval, concurrency, and recovery
- **Improved backup and recovery** — far more robust than one programmer's ad hoc script

Disadvantages of the Database Approach

The database approach is not free — it trades one set of problems for another, smaller but real:

Complexity

Designers, developers, DBAs, and end users all need to understand enough of the DBMS to use it well

Cost

DBMS software and the hardware it demands add significant up-front and ongoing expense

Reduced performance for narrow needs

A general-purpose DBMS can be slower than a program built for exactly one task

Higher impact of a failure

A centralized, shared database means an outage affects every dependent application at once

Why organizations adopt the database approach anyway

The redundancy and inconsistency problems only get worse as an organization grows, while the database approach's costs are largely one-time. A tiny, single-user tool with no sharing requirement may genuinely be better served by a plain file — it's a trade-off, not a universal law.

Key Takeaways

- A database is shared, organized, logically related data; a DBMS manages it; an information system is the database, DBMS, and applications together
- Databases quietly power banking, airlines, university records, telecom, sales, and HR systems
- File-based systems gave each application its own private files, causing redundancy, inconsistency, poor sharing, and program-data dependence
- The database approach replaces many private files with one centrally managed, shared pool of data accessed through a DBMS
- The database approach earns real advantages — consistency, integrity, security, standards, and recovery — at the cost of complexity, expense, and a bigger blast radius on failure